ESCOLA DE ROBOTICA E PROGRAMAÇÃO – SENAI CFP AFONSO GRECO

CURSO TECNICO DE TECNICO EM DESENVOLVIMENTO DE SISTEMAS

Padrões de projeto fabrica abstrata

Pedro Henrique Moura da Silva

Nova Lima - MG

2026

RESUMO

Este relatório técnico tem como objetivo apresentar a resolução da atividade prática referente à aplicação do padrão de projeto Abstract Factory, no contexto do curso de Desenvolvimento de Sistemas. O trabalho aborda a implementação de uma aplicação C# WPF funcional para demostrar o correto uso do padrão citado. Utilizando a linguagem C#, o projeto demonstra como o padrão Abstract Factory atende as demandas relacionadas a construção de classes funcionais em aplicações altamente escaláveis, evidenciando a separação de responsabilidades. Os resultados mostram que a abordagem adotada permite a fácil extensão para novos estilos com baixo acoplamento entre as classes.

SUMÁRIO

1. Introdução

2. Fundamentação teórica

2.1 Padrões de Projeto Criacionais

2.2 Abstract Factory

3. Desenvolvimento

3.1 Contexto do Problema

3.2 Estrutura do Projeto

3.3 Implementação

4. Resultado e Discussões

5. Considerações finais

6. Referências

1. Introdução

No desenvolvimento de software, um dos desafios recorrentes é a necessidade de construir sistemas que sejam independentes da forma como seus objetos internos são criados e compostos. Em especial, quando se que um sistema suporte múltiplas variantes de um mesmo componente (como temas visuais, sistemas operacionais ou configurações regionais), o acoplamento entre a lógica de negócio e a lógica de criação pode tornar a manutenção complexa.

O presente relatório documenta a resolução de uma atividade prática proposta na disciplina de Padrões de Projeto, do curso de Desenvolvimento de Sistemas. O objetivo principal foi

implementar um sistema de interface gráfica utilizando o padrão de projeto Abstract Factory, que pudesse ser um exemplo simples e de fácil explicação para exemplificação para os colegas de classe.

A atividade visou consolidar os conceitos de encapsulamento, polimorfismo e aderência aos princípios SOLID, demonstrando na prática como a utilização de padrões de projeto pode elevar a qualidade arquitetural de uma aplicação.

2. Fundamentação Teórica

A fundamentação teórica do método abstrato (ou Abstract Method) reside nos princípios da Programação Orientada a Objetos (POO) e em princípios de design de software que visam o desacoplamento e a extensibilidade.

OS pilares teóricos que sustentam esse conceito são:

Abstração e Modelagem

A abstração é um dos quatro pilares da POO e consiste no desafio de identificar características essenciais de um objeto do mundo real e transformá-las em uma classe.

O método abstrato é a representação mais pura desse pilar: ele modela um comportamento necessário, mas ignora os detalhes de como esse comportamento é realizado, deixando essa definição para um contexto específico.

Contrato e Obrigatoriedade

Um método abstrato atua como um contrato.

Quando uma classe base declara um método como abstrato, ela estabelece uma regra: todas as subclasses não abstratas que herdarem dessa base são obrigadas a fornecer uma implementação para esse método.

Isso garante que o sistema possa confiar que certas funcionalidades estarão disponíveis em todos os objetos de uma hierarquia, mesmo que as implementações sejam distintas.

Habilitação do Polimorfismo

A fundamentação teórica do método abstrato é o que permite o polimorfismo, que é a capacidade de um objeto ser flexível o suficiente para comportar-se de múltiplas formas.

Como a classe base define apenas a assinatura do método (o "o que fazer") e as subclasses definem a implementação (o "como fazer"), o programa pode detectar em tempo de execução a classe real do objeto e chamar o comportamento apropriado, mesmo que o tipo exato seja desconhecido no momento da compilação.

Princípios SOLID e Design Patterns

O método abstrato é a base para diversos princípios e padrões de projeto descritos nas fontes:

Princípio de Inversão de Dependência (DIP): "Abstração não deve depender de detalhes, detalhes devem depender de abstrações"

Programar voltado para métodos abstratos permite que o código dependa de abstrações, tornando o projeto flexível e fácil de estender sem quebrar o código existente.

Factory Method: Este padrão utiliza métodos abstratos para criar uma "fábrica" onde a classe base define a interface de criação, mas delega às subclasses a decisão de qual classe concreta instanciar.

Template Method: Define o esqueleto de um algoritmo em um método na superclasse, mas deixa que métodos abstratos (etapas específicas) sejam implementados pelas subclasses sem alterar a estrutura do algoritmo principal.

Características Técnicas

Inexistência de Instância: Uma classe que contém métodos abstratos deve ser obrigatoriamente abstrata e não pode ser instanciada diretamente.

Omissão de Implementação: O método abstrato não possui corpo na classe base; ele serve apenas como um "espaço reservado" ou hook para futuras implementações

Foco e Classificação

Esses padrões focam especificamente no processo de criação de classes e/ou objetos. Eles são subdivididos em dois escopos principais:

Escopo de Classe: Utilizam a herança no processo de criação para delegar a instanciação para subclasses (ex: Factory Method).

Escopo de Objeto: Utilizam a composição ou delegação para realizar a criação de forma mais dinâmica e flexível durante a execução (ex: Abstract Factory, Builder, Prototype e Singleton)

2.1 Padrões de Projeto Criacionais

Os padrões de projeto criacionais são uma categoria de padrões que se preocupam com o processo de criação de objetos. O principal objetivo desses padrões é fornecer mecanismos de criação que aumentem a flexibilidade e a reutilização do código já existente, desacoplando o sistema da forma como seus objetos são criados, compostos e representados.

2.2 Abstract Factory

O Abstract Factory (ou Fábrica Abstrata) é classificado como um padrão de projeto criacional com foco no escopo de objetos

Seu principal objetivo é fornecer uma interface que permita a criação de famílias de objetos relacionados ou dependentes sem a necessidade de especificar suas classes concretas

Um bom exemplo é uma aplicação de logística que em primeiro momento depende de veículos como caminhões para realizar seus serviços, considerando neste caso a parte inicial e fundamental da sua aplicação lida com transporte a maior parte do seu código pode estar alocada dentro da classe caminhão.

Entretanto isso cria gargalos na sua escalabilidade, se considerarmos que a sua aplicação se tornou popular e a sua empresa de logística foi recrutada para trabalhar com outros meios de transporte como o aéreo, você terá grandes dificuldades para implementar esse novo meio de transporte devido ao seu alto nível de acoplamento.

Como solucionar esse problema

O Abstract Factory propõe que você substitua a classe que cria o objeto de transporte focado no tipo do veiculo e a reescreva como uma subclasse que nesse caso depende diretamente de uma classe chamada por um método fabrica especial, no nosso exemplo seria a classe transporte. Existem algumas limitações relacionadas, agora só podemos retornar tipos diferentes de produtos se esses tiverem uma classe ou interface base em comum. O método fábrica na classe base deve ter seu tipo de retorno declarado como essa interface.

No nosso exemplo o método entregar é chamado independente do tipo de transporte utilizado, pois esse método será distinto para cada tipo de subclasse: caminhão, avião, navio entre outros...

Sua estrutura é composta por:

Abstract Factory organiza-se em torno de quatro elementos principais:

Produtos Abstratos: Declaram interfaces para um conjunto de produtos distintos, mas relacionados, que formam uma família.

Produtos Concretos: São as implementações específicas dos produtos abstratos, agrupadas por variantes.

Interface da Fábrica Abstrata: Declara os métodos de criação para cada um dos produtos abstratos.

Fábricas Concretas: Implementam os métodos da interface abstrata para produzir variantes específicas de produtos. Cada fábrica concreta cria apenas os produtos de sua respectiva variante.

3. Desenvolvimento

Para demonstrar o funcionamento do padrão abstract factory foi creiada a aplicação WPF exportação de documentos para Word e PDF, neste caso cada formato define uma família de objetos relacionados a um editor e um exportador, a fabrica abstrata IdocumentFactory declara os métodos de criação para os produtos da família.

As fabricas concretas WordDocumentFactory e PdfDocumentFactory retorna os objetos concretos correspondentes.

O código cliente trabalha apenas com interfaces IDocumentFactory, IDocumentEditor e IDocumentExporter. E não tem contato com as classes concretas o que permite trocar todo o comportamento do sistema apenas alterando a fábrica

3.1 Contexto do Problema:

A aplicação pode converter uma quantidades significativa de texto em formatos distintos escolhido de acordo com a necessidade do usuário é possível converter para Word e PDF

3.2 Estrutura:

- Produtos abstratos: Declaram interfaces para um conjunto de produtos distintos, mas relacionados que fazem parte de uma família de produtos

- Produtos concretos: São várias implementações de produtos abstratos, agrupados por variantes. Cada produto abstrato deve ser implementado em todas as variantes dadas.
- Interface: A fabrica abstrata declara um conjunto de métodos para criação e cada um dos produtos abstratos. Cada fabrica concreta corresponde a uma variante especifica de produtos e cria apenas aquelas variantes de produto
- Cliente: Este pode trabalhar com qualquer variante de produto/fábrica desde que ele se comunique com seus objetos via interfaces

4. Resultado e discussões

A aplicação do padrão Abstract Factory resultou em: Consistência garantindo que o editor e o exportador sejam do mesmo formato, Extensibilidade para adicionar quantos formatos sejam necessários sem alteração da interface principal e desacoplamento.

Flexibilidade: é possível adicionar um novo objeto específico apenas criando novas classes concretas, sem alterar o código existente.

Consistência: garante que os objetos criados por uma mesma superclasse são compatíveis entre si, alterando apenas suas características especificas a sua respectiva subclasse.

Manutenibilidade: a lógica de criação fica isolada em classes especializadas, facilitando correções e evoluções.

Análise Crítica:

Abordagem Sem o Padrão: Riscos e Limitações

Em um cenário onde o Abstract Factory não é utilizado, o código cliente geralmente recorre à instanciação direta de classes concretas via operador new

Isso gera diversos problemas técnicos descritos nas fontes:

Acoplamento Forte: O sistema torna-se dependente de classes específicas, o que reduz a flexibilidade e dificulta a reutilização do código

Código "Sujo" com Condicionais: Sem a fábrica, o programador frequentemente utiliza estruturas complexas de switch-case ou if-else espalhadas pelo sistema para decidir qual objeto criar

Isso viola o Princípio de Responsabilidade Única, pois a lógica de criação fica misturada à lógica de negócio

Inconsistência entre Famílias de Produtos: Existe o risco constante de o desenvolvedor misturar objetos de "famílias" diferentes que não deveriam colaborar entre si, resultando em falhas de compatibilidade

Violação do Princípio Aberto/Fechado: Toda vez que um novo produto ou variante é adicionado, é necessário modificar o código-fonte existente, o que aumenta o risco de introduzir bugs em funcionalidades que já estavam operando

Com o Abstract Factory

A implementação do padrão oferece uma solução estruturada para os problemas acima, mas impõe seu próprio conjunto de desafios:

Vantagens (Prós)

Garantia de Compatibilidade: O padrão assegura que os produtos obtidos de uma fábrica sejam compatíveis entre si, eliminando o erro humano de misturar variantes incompatíveis

Isolamento de Responsabilidade: Toda a lógica de criação é extraída para um único lugar, facilitando a manutenção e o entendimento da "receita" de fabricação dos objetos

Independência de Plataforma/Variante: O código cliente interage apenas com interfaces abstratas, permitindo que o sistema suporte novas famílias de produtos sem que o código principal precise ser alterado ou mesmo conheça as classes concretas

Desvantagens (Contras)

Explosão de Classes e Interfaces: A principal crítica ao padrão é que ele pode tornar o código excessivamente complicado

Para cada novo conjunto de produtos, é necessário criar uma nova interface de fábrica, fábricas concretas e interfaces para cada produto

Dificuldade de Extensão de Produtos: Embora seja fácil adicionar novas famírias (variantes), adicionar um novo tipo de produto à interface da fábrica abstrata exige alterar a interface base e todas as fábricas concretas já implementadas

Conclusão da Comparação

Enquanto a abordagem sem o padrão é mais rápida de implementar inicialmente, ela resulta em um sistema rígido e propenso a erros de compatibilidade

O Abstract Factory, por outro lado, é uma ferramenta de escalabilidade, recomendada quando o sistema lida com diversas famílias de produtos que precisam evoluir de forma independente das classes que os utilizam, aceitando o custo de uma estrutura de classes mais robusta e populada.

5. Considerações Finais

A utilização de padrões criacionais é recomendada para economizar recursos do sistema, como ao reutilizar conexões de banco de dados pesadas em vez de criá-las repetidamente, e para evitar o acoplamento forte entre o criador e os produtos concretos. Eles são ferramentas testadas para resolver problemas comuns de projeto de software usando princípios sólidos de orientação a objetos

Vantagens do padrão:

- Os produtos obtidos de uma fábrica são compatíveis entre as demais.
- Não há vinculo forte entre produtos concretos e o código cliente.
- Há responsabilidade única, sendo possível extrair o código de criação do produto para um lugar, tornando o de fácil manutenção.

Desvantagens:

- O código se torna altamente complexo uma vez que existem inúmeras novas classes junto com o padrão

6. Referências

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. **Design Patterns: Elements of Reusable Object-Oriented Software**. Indianapolis: Addison-Wesley Professional, 1994.

SANTANA, Rodrigo Gonçalves. **Design Patterns com C#: Aprenda padrões de projeto com os games**. São Paulo: Casa do Código, 2024. (ISBN 978-85-7254-051-3). Disponível também em versão MOBI (ISBN 978-85-7254-053-7).

.